

Efficient Non-Interactive Key Refresh with Multiple Independent Refreshers for Threshold Cryptography

Dragan Lambić*

Cpro Labs

University of Novi Sad, Faculty of Education, Sombor (Serbia)

Abstract

In this paper a novel key refresh architecture using multiple independent third-party refreshers, to eliminate the centralized trust required by single-dealer approaches, is presented. Each refresher independently maintains and refreshes a split of its share, distributing fragments to signing parties asynchronously without coordination. This eliminates the need for coordinated interaction required by committee-based refresh protocols, avoiding substantial communication overhead and synchronization complexity. The architectural separation, where signing parties never participate in refresh and refreshing parties never participate in signing, enables efficient proactive security without disrupting operational availability. The proposed design does not require all refreshers to participate in each refresh operation. Unavailable refreshers are not excluded from the protocol, only malicious ones that deviate from protocol rules. This provides graceful degradation: the system maintains confidentiality even when all refreshers are compromised (requiring only one honest signing party) and achieves proactive security with minimal participation (requiring only one honest refresher between compromise events). This approach extends the period between costly on-chain key rotations by maintaining security through continuous off-chain refresh. Key rotation should be carried out when very few honest refreshers remain. Security under various adversarial scenarios is proven, including malicious refreshers, cross-epoch compromises, and denial-of-service attacks, demonstrating $O(k)$ communication per refresh for k participating refreshers versus $O(n^2)$ for committee-based protocols. The proposed approach is particularly suited for high-availability systems requiring frequent or continuous key refresh, including cryptocurrency wallets and distributed key management.

Keywords: Key refresh, Threshold cryptography, Proactive security, Blockchain

1. Introduction

Threshold cryptography has emerged as a fundamental technique for securing critical systems by distributing trust across multiple parties [1]. Rather than storing a private key at a single location, threshold schemes generate the key in distributed shares (or derive shares from an existing key), which are held by different parties and require a threshold number to collaborate for cryptographic operations. This approach has proven invaluable for applications ranging from cryptocurrency custody [2-5] to secure multi-party computation [6], key management infrastructure [7], and distributed certificate authorities [8]. Threshold cryptography distributes control over a secret among multiple parties so that no single entity can act unilaterally, while a qualified subset can jointly perform cryptographic operations [24]. Within this design space, two-party multi-party computation (2PC-MPC) represents a specialized form of threshold secret

sharing typically aimed at securing the user-network relationship, ensuring that both sides retain full cryptographic control even in the presence of a fully corrupted counterparty. 2PC-MPC eliminates honest majority assumptions by using a 2-of-2 threshold, providing user sovereignty (users control one key share directly) while distributing trust between user and network. This makes it attractive for wallet applications where regulatory compliance or user control requirements prevent fully delegating key management to a network. However, these benefits come at the cost of reduced structural redundancy and verifiability, as many safeguards available in larger threshold systems must be replaced by bilateral coordination and off-protocol assumptions. As a result, subtle failures in key generation, refresh, and lifecycle management become more difficult to prevent or detect.

Existing two-party (or multi-party) key-management protocols based on additive distributed key generation (DKG) can exhibit structural weaknesses when implemented without strong coordination, commitment, and lifecycle guarantees. In the absence of atomic execution and transcript binding, key refresh operations may incorporate inconsistent randomness across parties, leading to

*Corresponding author

Email address: draganposao@yahoo.com (Dragan Lambić)

desynchronized or invalid key states. Similarly, additive constructions that lack robust commit-and-verify mechanisms may permit a participant to bias the resulting secret key by adversarially selecting its share without readily detectable evidence. Refresh procedures can also be vulnerable to replay or reuse of randomness if session integrity is not strictly enforced, potentially undermining forward secrecy. Share migration to a new party is typically non-atomic and cannot cryptographically guarantee secure deletion of obsolete shares, leaving residual exposure risks. Finally, where systems lack an auditable key-share lifecycle, key creation, refresh, migration, and destruction events may be opaque, reducing accountability and complicating failure diagnosis. Together, these factors can diminish the robustness and usability of threshold wallets and expose users to subtle, often silent failures [31, 32]. A critical challenge in long-lived threshold systems is proactive security: defending against adversaries who gradually compromise different parties over time. Herzberg et al. [9] introduced proactive secret sharing (PSS) to address this threat by periodically refreshing shares. The fundamental security property of PSS is that an adversary must compromise a threshold number of parties within a single refresh epoch to recover the secret. Shares compromised in previous epochs become useless after refresh, under the assumption that honest parties delete their shares, preventing an adversary who compromises different parties across multiple epochs from reconstructing the secret. This significantly strengthens security compared to static secret sharing, where any t -out-of- n shares suffice for reconstruction regardless of when they were obtained.

Despite significant theoretical advances in proactive secret sharing [9-14], practical deployment faces persistent obstacles. Existing refresh mechanisms generally fall into two categories, each with substantial limitations. Committee-based refresh protocols [13-17] require multiple parties to interact, generating significant communication overhead (typically $O(n^2)$ or $O(n^3)$ where n is the number of parties), synchronization complexity, and operational disruption during refresh operations. Single-dealer approaches [18] achieve non-interactive refresh but introduce centralized trust assumptions, creating single points of failure where a malicious or compromised dealer can undermine the security of the entire system.

These limitations become particularly acute in scenarios requiring frequent refresh. Systems with high security requirements may need daily or even more frequent refresh to minimize exposure windows. Applications involving credential transfer or time-bounded access control require refresh upon each transfer or expiration event. In such settings, the resource costs and coordination overhead of committee-based protocols become prohibitive, while single-dealer approaches present unacceptable security risks. This paper introduces a novel key refresh architecture for two-party computation, extensible to multi-party computation, that achieves efficient, non-interactive

refresh without centralized trust assumptions. Here, non-interactive means that signing parties do not coordinate during refresh: refreshers operate independently without synchronization, and signing parties apply updates without interacting with each other. Our approach builds upon recent advances in offline refresh [19] and role-based secret sharing [18], but differs fundamentally in its architectural design. Specifically, while Kondi et al. [19] introduced offline refresh for backup scenarios with a single auxiliary device, and He et al. [18] demonstrated non-interactive refresh with a centralized dealer, our approach employs multiple independent third-party refreshers that operate autonomously without coordination. This combination of non-interactive operation (similar to [18]) with decentralized trust (avoiding the single-dealer limitation) represents a novel contribution to the proactive security landscape.

Our architecture is based on complete role separation: signing parties never participate in refresh operations, and refreshing parties never participate in signature generation. Each third party independently maintains its share as a split and periodically re-randomizes this split, distributing updated fragments to both signing parties. Critically, refreshers operate autonomously—no coordination among refreshers is required, and refresh proceeds asynchronously without disrupting the availability of the signing system. This design provides graceful degradation under partial failures (system remains secure even when all refreshers except one are compromised) and enables continuous refresh at arbitrary frequencies while maintaining a simple 2-of-2 signing threshold between the two active parties.

1.1. Motivation and Design Requirements

The motivation for this work stems from fundamental limitations in existing key refresh mechanisms. State-of-the-art committee-based protocols such as CHURP [13], DyCAPS [14], and their recent optimizations [15-17] achieve strong security properties but require $O(n^2)$ to $O(n^3)$ communication complexity and coordinated interaction among committee members. For instance, DyCAPS requires all committee members to participate in bivariate polynomial generation during handoff [14], while CHURP requires $O(n^2)$ off-chain communication even in the optimistic case [13]. Recent work on two-party threshold ECDSA with proactive security [20] achieves efficient refresh between two parties but requires both parties to participate actively in the refresh protocol, creating operational overhead and limiting refresh frequency.

In contrast, He et al. [18] recently demonstrated that non-interactive refresh can be achieved using Threshold Homomorphic Encryption (ThHE), where a dealer broadcasts encrypted refresh arrays enabling $O(n)$ communication complexity. However, their architecture relies on a single trusted dealer—a centralization point that contradicts the fundamental goal of distributed trust in threshold cryptography. A malicious or compromised dealer can potentially undermine security by generating biased refresh

values or selectively providing refresh information to subsets of participants.

Our approach addresses both limitations simultaneously. Like He et al. [18], we achieve non-interactive refresh where participants do not need to coordinate during refresh operations. However, unlike their centralized dealer model, we distribute refresh authority among multiple independent third parties. Each refresher operates autonomously, and the system tolerates malicious behavior by individual refreshers without compromising security. This architectural innovation satisfies several critical design requirements:

- **Efficiency:** Minimal computational and communication overhead to enable frequent refresh
- **Non-disruption:** Refresh should not interfere with signing operations or require system downtime
- **Decentralized trust:** No single entity should control refresh operations
- **Asynchronous operation:** Refreshers should operate independently without coordination
- **Continuous availability:** The system should support refresh at arbitrary frequencies without degradation

These requirements are motivated by diverse practical scenarios. High-security systems may require frequent (e.g., hourly) proactive refresh to minimize the window of vulnerability [9]. Transferable credentials or digital assets require immediate share invalidation upon ownership transfer to prevent previous owners from retaining access. Time-bounded access control systems [21] require shares to automatically expire after specified periods. High-availability systems in cloud or distributed environments [22] require refresh to proceed without disrupting operational signing capacity. Resource-constrained devices [23] may be unable to participate in complex refresh protocols while maintaining their primary signing functions.

To our knowledge, no existing system simultaneously satisfies all five design requirements. Committee-based approaches fail the efficiency and non-disruption requirements due to coordination overhead. Single-dealer approaches fail the decentralized trust requirement. Our architecture is the first to achieve all five properties through the novel combination of multiple independent refreshers with strict role separation between signing and refreshing parties.

The remainder of this paper is organized as follows. Section 2 surveys related work in threshold cryptography and proactive secret sharing. Section 3 presents our system model, architectural design, and detailed protocol specifications for key generation, refresh, and synchronization. Section 4 provides security analysis including confidentiality proofs, proactive security guarantees, collusion resistance, and denial-of-service analysis. Section 5 compares our approach with existing schemes and analyzes

communication and computational complexity. Section 6 discusses limitations and proposes a hybrid model where refreshers participate in signing.

2. Related Work

Our work builds upon extensive research in threshold cryptography and proactive secret sharing. While our approach is presented in the context of additive sharing compatible with threshold ECDSA, the architectural principles apply equally to other threshold signature schemes. Proactive secret sharing (PSS), introduced by Herzberg et al. [9], extends traditional secret sharing with periodic share refresh to defend against mobile adversaries who compromise different parties over time. The key insight is that an adversary must compromise a threshold number of parties within a single epoch to recover the secret; shares obtained from previous epochs become useless after refresh. This addresses the fundamental limitation of static secret sharing where an adversary who eventually compromises t parties—even across different time periods—can reconstruct the secret.

Mobile proactive secret sharing (MPSS) [10] extended PSS to dynamic groups where committee membership changes over time. Baron et al. [11] optimized communication complexity for dynamic groups, achieving $O(n^2)$ communication. Eldefrawy et al. [12] addressed dishonest majority scenarios, an important consideration for adversarial environments. Günther et al. [25] presented practical asynchronous proactive secret sharing, demonstrating that PSS can be deployed in real-world asynchronous networks without timing assumptions. These protocols typically require coordinated refresh where committee members interact to generate and distribute new shares. While achieving strong security properties, this coordination requirement imposes significant communication overhead and operational complexity, limitations our work addresses through independent third-party refreshers.

Recent work has focused on dynamic-committee proactive secret sharing (DPSS) where committee membership evolves through handoff protocols. CHURP [13] introduced an efficient dynamic-committee PSS with $O(n)$ on-chain and $O(n^2)$ off-chain communication in the optimistic case, enabling committee rotation for blockchain applications. DyCAPS [14] achieved efficient asynchronous DPSS using bivariate polynomials for committee handoff, with experimental results showing handoff between 4-node committees in 1.3 seconds. Building on DyCAPS, Hu et al. [15] recently presented optimistic asynchronous DPSS that reduces message complexity in optimistic execution paths, achieving better performance when no Byzantine faults occur. Bandrupalli et al. [16] further optimized batched asynchronous DPSS using lightweight symmetric cryptography, demonstrating scalability beyond tens of parties. Yurek et al. [17] presented Long Live the Honey Badger, achieving optimal fault tolerance and batch amortization without trusted setup. These protocols excel at large-

scale committee rotation and achieve optimal resilience in Byzantine settings. However, they fundamentally assume that committee members both sign and refresh, requiring coordinated handoff procedures. Our architecture differs by separating these roles: signing parties maintain stable long-term positions while refreshing parties operate independently, enabling continuous refresh without the coordination overhead inherent in committee-based approaches.

Most closely related to our work is research on offline refresh and non-interactive mechanisms. Kondi et al. [19] presented "Refresh When You Wake Up," enabling proactive threshold wallets where offline devices can catch up asynchronously. Their protocol targets a 2-of-3 scenario where two parties actively sign while a third device (such as cold storage backup) remains offline and updates when convenient. This pioneering work demonstrated that refresh can proceed without requiring all parties to be simultaneously online. While conceptually related, critical differences distinguish our approach. First, Kondi et al. [19] focus on a single offline device for backup/recovery scenarios, whereas we employ multiple independent refreshers for enhanced security and availability. Second, their system operates periodically for proactive security, while our architecture supports continuous refresh for applications requiring frequent share invalidation. Third, their offline device eventually participates in signing (for recovery), while our refreshers never participate in signing operations, maintaining strict role separation.

He et al. [18] recently presented role-based efficient PSS with user revocation using Threshold Homomorphic Encryption (ThHE). Their approach achieves $O(n)$ communication complexity through a centralized dealer that broadcasts encrypted refresh arrays, enabling non-interactive refresh for participants. This represents significant progress in refresh efficiency and demonstrates the viability of non-interactive approaches.

However, their architecture relies on a single trusted dealer coordinating all refresh operations—a centralization point that creates security vulnerabilities. If the dealer is compromised or behaves maliciously, the security of the entire system is compromised. Our approach eliminates this centralized trust assumption by distributing refresh authority among multiple independent third parties. Each refresher operates autonomously, and the system tolerates malicious behavior by individual refreshers without compromising overall security. This decentralization is achieved while maintaining the non-interactive properties demonstrated by He et al., combining the benefits of both approaches.

Existing production implementations [26, 27, 28, 29, 30] typically treat refresh as an occasional maintenance operation rather than a continuous capability. Refresh often requires coordinated downtime or imposes performance degradation during the refresh period. Our architecture addresses these limitations by enabling continuous background refresh that operates transparently without affecting signing operations, making frequent refresh practical

for production deployments. To our knowledge, this is the first work to propose multiple independent third-party refreshers operating alongside stable signing parties, and the first to formalize the architectural principle of complete role separation between signing and refreshing parties. This architectural innovation enables new applications requiring frequent or continuous refresh while maintaining strong security properties and practical efficiency.

3. Proposed Method

This section presents our key refresh architecture, including the system model, architectural design principles, and protocol specifications for key generation, refresh, and synchronization.

3.1. System Model and Assumptions

3.1.1. Participants and Roles

Our system involves two distinct types of participants, each with specific roles and responsibilities:

- **Signing Parties:** Party 1 holds share x_1 and Party 2 holds share x_2 . Both participate in all signature generation operations but never participate in refresh operations. Each party may be implemented as a single entity or distributed system (e.g., a threshold network). In wallet applications, Party 1 is typically the user-controlled party while Party 2 may be a network service. From the protocol's perspective, each appears as a single logical party.

- **Third-Party Refreshers ($R_1, R_2, \dots, R_m$):** Each refresher R_i holds a share x_{i+2} of the secret key and randomly splits it in two parts. Refreshers periodically or continuously re-randomize their split and distribute updates to both signing parties. Refreshers never participate in signature generation.

This role separation enables continuous refresh without disrupting signing availability and allows each role to be optimized independently. The secret key x is distributed additively as:

$$x = x_1 + x_2 + \sum_{i=1}^m x_{i+2} \quad (1)$$

Each third party R_i maintains its share x_{i+2} as a split

$$x_{i+2} = r_i + s_i \quad (2)$$

where r_i is randomly chosen and

$$s_i = x_{i+2} - r_i. \quad (3)$$

The value r_i is distributed to Party 1, while s_i is distributed to Party 2. Thus, Party 1 effectively holds

$$x_1^{\text{eff}} = x_1 + \sum_{i=1}^m r_i \quad (4)$$

And Party 2 effectively holds

$$x_2^{\text{eff}} = x_2 + \sum_{i=1}^m s_i = x_2 + \sum_{i=1}^m (x_{i+2} - r_i). \quad (5)$$

3.1.2. Network Model and Refresher Availability

We assume an asynchronous network model where messages may experience arbitrary but finite delays. There is no global clock, and parties cannot rely on timing assumptions for correctness. However, we assume the existence of authenticated communication channels between:

- Each refresher R_i and Party 1,
- Each refresher R_i and Party 2,
- Party 1 and Party 2 (for signing operations).

Communication channels provide message authentication and integrity but not necessarily confidentiality (though confidentiality can be added through standard encryption). Messages may be delayed, reordered, or duplicated, but authenticated messages cannot be forged or modified by an adversary.

Refresher Availability. A critical feature of our architecture is that not every refresher needs to provide a new split during every refresh cycle. If refresher R_i is temporarily unavailable, offline, or otherwise unable to perform a refresh operation, the signing parties continue using the most recent split (r_i, s_i) previously provided by that refresher.

This design provides several important properties:

- **Fault Tolerance.** The system continues operating normally even if some refreshers are temporarily unavailable. Only refreshers that are currently online and operational perform refresh operations. A single available refresher is sufficient for key refresh.
- **Asynchronous Operation.** Refreshers operate on independent schedules without coordination. There is no requirement for refreshers to synchronize or perform refresh simultaneously.
- **Gradual Refresh.** As refreshers come back online or complete refresh operations at different times, the overall security of the system gradually improves as more shares receive fresh splits.
- **No Single Point of Failure.** Unlike approaches requiring all parties to participate in refresh [13-17] or a single dealer to be available [18], our system tolerates arbitrary subsets of refreshers being unavailable without disrupting operations.

Formally, at any point in time, Party 1 holds

$$x_1^{\text{eff}} = x_1 + \sum_{i=1}^m r_i^{(e_i)} \quad (6)$$

where $r_i^{(e_i)}$ denotes the value of r_i from the most recent refresh epoch e_i performed by refresher R_i .

Different refreshers may have different current epoch numbers $e_i \neq e_j$, reflecting their independent operation. The invariant

$$x_1^{\text{eff}} + x_2^{\text{eff}} = x \quad (7)$$

holds regardless of which subset of refreshers has recently refreshed, as long as both signing parties have applied the same sequence of updates from each refresher.

3.1.3. Threat Model and Security Assumptions

We consider a computationally bounded probabilistic polynomial-time (PPT) adversary A who can adaptively corrupt parties during protocol execution. The adversary has the following capabilities:

- **Adaptive Corruption:** A can choose which parties to corrupt at any time during protocol execution, based on information observed so far.
- **Byzantine Behavior:** Corrupted parties may deviate arbitrarily from the protocol specification, including refusing to send messages, sending incorrect values, or colluding with other corrupted parties.
- **Complete State Access:** Upon corrupting a party, A learns all internal state of that party, including secret shares, split components, and any stored randomness.
- **Network Control:** A can delay, reorder, or drop messages between honest parties (asynchronous network model), but cannot forge or modify messages on authenticated channels.
- **Collusion:** Corrupted parties may collude and share information.

Security Threshold: The system maintains security if the adversary does not simultaneously corrupt both signing parties (Party 1 and Party 2) within the same refresh epoch. Formally, let epoch e be the time interval between refresh operations. Security holds if:

- At any time t , at least one of {Party 1, Party 2} remains honest, OR
- If both parties are corrupted at different times t_1 and t_2 , then at least one refresh operation (by any refresher) completes between t_1 and t_2 , and honest parties securely erase their old share split components after applying the refresh

Note that the security guarantee depends on at least one refresher performing a refresh operation between corruption events, not on all refreshers refreshing. This reflects the fault-tolerant nature of our design where partial refresh (by a subset of available refreshers) still provides proactive security benefits.

Refresher Trust Assumptions: No trust assumptions are required regarding refresher honesty or availability. Confidentiality of the secret key x is maintained even if all m refreshers are compromised, as long as at least one signing party remains honest (inherent to the 2-of-2 threshold). Proactive security requires only that at least one honest refresher can perform refresh between compromise events

of the two signing parties and keep its underlying share secret. The system degrades gracefully: with k honest refreshers ($0 \leq k \leq m$), confidentiality is always maintained; proactive security is provided when $k \geq 1$. Refreshers operate independently without requiring mutual trust or coordination.

Third-Party Refresher Corruption: The adversary may corrupt any number of third-party refreshers. Corrupting refresher R_i reveals x_{i+2} , r_i , and s_i but does not reveal x_1 , x_2 , or the shares of other refreshers. A malicious refresher can:

- Refuse to send refresh updates (denial of service)
- Send different updates to Party 1 and Party 2 (equivocation)
- Choose biased randomness for refresh

However, these attacks do not compromise the secret key x as long as at least one signing party remains honest. Equivocation can be detected through the consistency verification mechanisms described in Section 3.6.

Authenticated Channels: Communication channels provide message authentication, preventing adversaries from forging messages claiming to be from honest parties.

Secure Erasure Assumption: Following standard assumptions in proactive secret sharing [9, 10, 19], we assume that honest parties can securely erase old key material after completing a refresh. Specifically, after Party 1 applies a refresh update from refresher R_i at epoch e (receiving new value $r_i^{(e)}$), it erases the previous value $r_i^{(e-1)}$. Similarly, Party 2 erases old s_i values after applying updates. We emphasize that this assumption is about erasing old split components (old r_i , s_i values), not about erasing the shares x_1 , x_2 , or x_{i+2} themselves, which remain constant. While secure deletion is a well-known challenge without a general cryptographic solution, this assumption is standard in the PSS literature and can be strengthened through deployment practices such as trusted hardware, frequent refresh intervals, or time-bounded key material.

Cryptographic Assumptions: Our protocol relies on standard cryptographic assumptions:

- The discrete logarithm problem is hard in the chosen group (for ECDSA-based instantiation)
- The existence of collision-resistant hash functions
- IND-CPA secure encryption
- The existence of secure authenticated encryption schemes

3.2. Architectural Design

3.2.1. Role Separation Principle

The core architectural innovation of our approach is complete separation between signing and refreshing roles. This separation provides several critical advantages:

- **Non-Disruption:** Signing operations proceed normally during refresh, with no performance degradation or coordination overhead. Even if multiple refreshers are simultaneously performing refresh operations, signing continues uninterrupted.
- **Simplicity:** Each role can be optimized independently. Signing parties implement efficient 2-of-2 threshold signatures without refresh logic, while refreshers implement simple re-randomization of splits without signing logic.
- **Continuous Operation:** Refresh can occur at arbitrary frequencies (even continuously) without impacting signing availability. Individual refreshers can operate on independent schedules.
- **Partial Refresh Capability:** If only a subset of refreshers are available at any given time, they can still perform refresh operations independently. The system does not require all refreshers to be online simultaneously, providing robustness against temporary failures or network partitions.

This contrasts sharply with committee-based approaches [13-17] where the same parties both sign and refresh, requiring careful coordination to avoid disrupting operations. It also differs from single-dealer approaches [18] where the dealer is a distinct entity but represents a single point of trust and a single point of failure, if the dealer is unavailable, no refresh can occur.

3.2.2. Share Split Strategy

Each refresher R_i maintains its share x_{i+2} as a split (r_i, s_i) where:

$$r_i \in \mathbb{Z}_q \quad (\text{chosen uniformly at random}) \quad (8)$$

$$s_i = x_{i+2} - r_i \bmod q \quad (9)$$

The value r_i is securely communicated to Party 1, and s_i is securely communicated to Party 2. This distribution strategy ensures that:

- **Confidentiality:** Neither Party 1 nor Party 2 learns x_{i+2} , as each receives only one component of the split.
- **Correctness:** The sum of all components equals the original secret:

$$x_1 + \sum r_i + x_2 + \sum s_i = x_1 + x_2 + \sum (r_i + s_i) = x_1 + x_2 + \sum x_{i+2} \quad (10)$$

- **Refresh Capability:** Refresher R_i can refresh by choosing new r'_i , computing

$$s'_i = x_{i+2} - r'_i \quad (11)$$

and communicating the new values.

- **Independent Operation:** Different refreshers can refresh at different times. If refresher R_j is unavailable, signing parties continue using its most recent split (r_j, s_j) while other refreshers update their splits independently.

3.2.3. Synchronization Strategy

A critical challenge in any refresh protocol is ensuring that all honest parties maintain consistent views of the current epoch and share split values. Our approach uses epoch identifiers and sequence numbers:

- Each refresh operation by R_i is assigned a monotonically increasing sequence number seq_i .
- Refresh messages include $(i, \text{seq}_i, R'_i, S'_i, \text{Enc}_{\text{pk}_1}(r'_i), \text{Enc}_{\text{pk}_2}(s'_i), \text{signature})$ where i identifies the refresher, seq_i is the sequence number, R'_i and S'_i are the new public commitments, and the encrypted values are sent to the respective parties.
- Parties maintain a table tracking the latest applied sequence number for each refresher: $\text{last_seq}[i]$ represents the most recent update from R_i .
- Parties apply updates idempotently: if $\text{seq}_i \leq \text{last_seq}[i]$, the update is ignored as already applied.

This mechanism ensures eventual consistency: if both Party 1 and Party 2 eventually receive and apply all refresh messages up to sequence numbers $\{\text{seq}_1, \text{seq}_2, \dots, \text{seq}_m\}$, they will hold consistent shares that sum to the correct secret key x . Crucially, refreshers can have different current sequence numbers ($\text{seq}_1 \neq \text{seq}_2 \neq \dots \neq \text{seq}_m$) reflecting their independent operation schedules. Section 3.6 provides detailed synchronization protocols and analysis.

3.3. Protocol: Distributed Key Generation

The key generation protocol establishes the initial distribution of shares among all parties. Since our system uses additive secret sharing with $m+2$ parties total (Party 1, Party 2, and m refreshers), all parties participate in a distributed key generation protocol without requiring a trusted dealer. Each party's share remains secret throughout the protocol. The complete protocol is presented in Algorithm 1.

Verification mechanism: This protocol ensures strong security and verifiability through a two-layer verification mechanism. First, in Phase 4, all parties publicly verify that each refresher's commitments are consistent: $R_i + S_i = Y_{i+2}$. This ensures that the refresher has committed to split components that sum to their claimed share. Second, in Phase 5, Party 1 and Party 2 privately verify that the encrypted values they received match the public commitments: Party 1 checks $r_i \cdot G = R_i$ and Party 2 checks $s_i \cdot G = S_i$. If a refresher passes Phase 4 but fails Phase 5, it means the refresher sent different encrypted values than

what it publicly committed to, which is detected and the refresher is excluded.

The key security properties are: (1) No party learns any other party's secret share x_j , as shares are never revealed, only commitments (R_i, S_i) are broadcast. (2) Malicious refreshers are identified and excluded at two stages: inconsistent public commitments in Phase 4, and inconsistent encrypted values in Phase 5. (3) The protocol provides public verifiability, anyone can verify that $R_i + S_i = Y_{i+2}$ for all honest refreshers. (4) The protocol provides graceful degradation, continuing successfully with the honest subset H of refreshers rather than aborting when malicious behavior is detected. (5) The final consistency check in Phase 6 ensures that the effective shares held by Party 1 and Party 2 correctly sum to the secret key x .

Enhanced Security with Homomorphic Encryption: The protocol can be further strengthened by using homomorphic encryption schemes such as Paillier encryption. With homomorphic properties, refreshers can provide zero-knowledge proofs that the encrypted ciphertexts correctly encrypt values whose discrete logarithms correspond to the public commitments R_i and S_i . Specifically, a refresher can prove that $\text{Enc}_{\text{pk}_1}(r_i)$ encrypts a value r such that $r \cdot G = R_i$, and similarly for Party 2. This eliminates the need for the complaint mechanism in Phase 5, as malicious refreshers would be unable to construct valid proofs for inconsistent encryptions. Such proofs can be constructed using standard techniques combining Schnorr-style proofs with proofs of correct encryption under homomorphic schemes.

The encryption keys $(\text{pk}_1, \text{pk}_2)$ established in Phase 2 are subsequently used for all future refresh operations described in Section 3.5. The public commitments (R_i, S_i) serve a dual purpose: they enable public verification of consistency during key generation, and they can be updated during refresh operations to maintain verifiability throughout the protocol's lifetime.

Signing Operations: After successful key generation, Party 1 and Party 2 can generate threshold signatures using their effective shares $x_1^{\text{eff}} = x_1 + \sum_{i \in H} r_i$ and $x_2^{\text{eff}} = x_2 + \sum_{i \in H} s_i$. Any standard 2-of-2 threshold signature protocol for the chosen signature scheme (e.g., Lindell [2], GG20 [4], or CGGMP [5] for ECDSA) can be applied directly. The signing protocol operates solely between Party 1 and Party 2; refreshers do not participate in signature generation, maintaining strict role separation. Signatures are valid under the public key Y and indistinguishable from signatures generated using the secret key x directly.

3.4. Protocol: Key Refresh

The key refresh protocol allows third-party refreshers to independently re-randomize their share splits, distributing updated split components to both signing parties. The protocol structure closely follows the key generation protocol (Section 3.3, Phases 2-6), with the following key differences:

Differences from Key Generation:

Algorithm 1: Distributed Key Generation Protocol

Participants: Party 1, Party 2, $R_1, R_2, \dots, R_m$ (total $m+2$ parties)

Prerequisites: Elliptic curve parameters (G, q) from chosen signature scheme (e.g., secp256k1)

Output: Party 1 receives $(x_1, \{r_i\}_{i \in H})$, Party 2 receives $(x_2, \{s_i\}_{i \in H})$, Refresher R_i receives (x_{i+2}, r_i, s_i) for $i \in H$; Public key Y published

Phase 1: Public Key Generation

1. Each party $j \in \{1, 2, 3, \dots, m+2\}$ independently samples a random secret share: $x_j \leftarrow \mathbb{Z}_q$. The share x_j is kept secret and never revealed to any other party.
2. Each party j computes and broadcasts its public key contribution: $Y_j = x_j \cdot G$ where G is the generator of the elliptic curve group.
3. All parties verify that each public key contribution is valid, $Y_j \neq 0$ (point at infinity) for all $j \in \{1, 2, \dots, m+2\}$.
4. All parties compute the preliminary joint public key: $Y^{temp} = Y_1 + Y_2 + Y_3 + \dots + Y_{m+2}$
5. All parties verify that $Y^{temp} \neq 0$, ensuring the secret key is non-zero.

Phase 2: Encryption Key Establishment

6. Party 1 generates an encryption key pair for receiving split components $(sk_1, pk_1) \leftarrow \text{KeyGen}()$ and broadcasts pk_1 (public encryption key). Party 2 generates $(sk_2, pk_2) \leftarrow \text{KeyGen}()$ and broadcasts pk_2 .

Phase 3: Share Splitting and Commitment

7. For each refresher $i = 1$ to m :
 - a. Refresher R_i samples random split component: $r_i \leftarrow \mathbb{Z}_q$
 - b. Compute complementary split component: $s_i = x_{i+2} - r_i \mod q$
 - c. Compute and broadcast public commitments to split components: $R_i = r_i \cdot G$, $S_i = s_i \cdot G$, broadcast (R_i, S_i)
 - d. Encrypt r_i with Party 1's public key: $\text{ciphertext}_1^{(i)} = \text{Enc}_{pk_1}(r_i)$
 - e. Encrypt s_i with Party 2's public key: $\text{ciphertext}_2^{(i)} = \text{Enc}_{pk_2}(s_i)$
 - f. Send $\text{ciphertext}_1^{(i)}$ to Party 1 over authenticated channel
 - g. Send $\text{ciphertext}_2^{(i)}$ to Party 2 over authenticated channel

Phase 4: Public Verification of Commitments

8. For each refresher $i = 1$ to m , all parties perform public verification:
 - a. Verify that the split commitments sum to the public key contribution: Check $R_i + S_i = Y_{i+2}$
 - b. If $R_i + S_i \neq Y_{i+2}$: Refresher R_i has provided inconsistent commitments. Mark R_i as malicious and exclude from protocol. Continue with remaining refreshers.

Phase 5: Private Verification by Signing Parties

9. Party 1 performs private verification:
 - a. For each refresher i that passed Phase 4 verification: Decrypt $r_i = \text{Dec}_{sk_1}(\text{ciphertext}_1^{(i)})$. Verify $r_i \cdot G = R_i$ (the broadcasted commitment).
 - b. If $r_i \cdot G \neq R_i$ for any refresher i : Refresher R_i sent inconsistent encrypted value. Party 1 broadcasts complaint $(i, R_i, r_i, \text{proof})$. All parties mark R_i as malicious and exclude from protocol.
10. Party 2 performs private verification:
 - a. For each refresher i that passed Phase 4 verification: Decrypt $s_i = \text{Dec}_{sk_2}(\text{ciphertext}_2^{(i)})$. Verify $s_i \cdot G = S_i$ (the broadcasted commitment).
 - b. If $s_i \cdot G \neq S_i$ for any refresher i : Refresher R_i sent inconsistent encrypted value. Party 2 broadcasts complaint $(i, S_i, s_i, \text{proof})$. All parties mark R_i as malicious and exclude from protocol.
11. Define the honest refresher set: $H = \{i \in \{1, 2, \dots, m\} : R_i \text{ passed all verifications}\}$
12. All parties recompute the final public key using only honest refreshers: $Y = Y_1 + Y_2 + \sum_{i \in H} Y_{i+2}$
13. Verify that $Y \neq 0$. If $Y = 0$, abort the protocol.

Phase 6: Final Consistency Check

14. Party 1 computes its effective share using only honest refreshers: $x_1^{eff} = x_1 + \sum_{i \in H} r_i \mod q$
15. Party 2 computes its effective share using only honest refreshers: $x_2^{eff} = x_2 + \sum_{i \in H} s_i \mod q$
16. Party 1 computes and broadcasts: $Y_1^{eff} = x_1^{eff} \cdot G$
17. Party 2 computes and broadcasts: $Y_2^{eff} = x_2^{eff} \cdot G$
18. All parties verify final consistency: Check that $Y_1^{eff} + Y_2^{eff} = Y$. If verification fails, abort the protocol.
19. If all verifications pass, the protocol succeeds. Each party stores: Party 1: $(x_1, \{r_i\}_{i \in H}, sk_1, pk_1)$; Party 2: $(x_2, \{s_i\}_{i \in H}, sk_2, pk_2)$; Honest refresher R_i ($i \in H$): $(x_{i+2}, r_i, s_i, R_i, S_i)$; Public values known to all: $Y, \{Y_1, Y_2, \{Y_j\}_{j=3}^{m+2}\}, \{R_i, S_i\}_{i \in H}$; Malicious refreshers ($i \notin H$) are permanently excluded from the system.

Figure 1: Algorithm 1

1. **Partial Participation:** Not all refreshers need to participate in a given refresh operation. If refresher R_i is unavailable or chooses not to refresh, signing parties continue using its most recent valid split components (r_i, s_i) . The set of participating refreshers $P \subseteq H$ may be any non-empty subset of honest refreshers.
2. **No Public Key Recomputation:** The public key Y remains constant. Only the split components change, not the underlying shares x_{i+2} .
3. **Encryption Keys Already Established:** Parties use the existing encryption keys (pk_1, pk_2) from key generation. No new key generation phase is needed.
4. **Sequential Numbering:** Each refresher R_i maintains a monotonically increasing sequence number seq_i for its refresh operations, enabling parties to track and apply updates idempotently.
5. **Handling Malicious Behavior:** If refresher R_i is detected as malicious during refresh (fails verification in Phase 4 or Phase 5), it is permanently marked as malicious and excluded from all subsequent refresh operations. From that point forward, signing parties continue using R_i 's last known valid split components before it became malicious. The honest set H is updated: $H \leftarrow H \setminus \{i\}$.

Protocol Execution:

For each participating refresher $R_i \in P$:

1. Generate new split: $r'_i \in \mathbb{Z}_q$, $s'_i = x_{i+2} - r'_i \mod q$.

2. Compute commitments: $R'_i = r'_i \cdot G$, $S'_i = s'_i \cdot G$.
3. Broadcast: $(i, \text{seq}_i + 1, R'_i, S'_i)$.
4. Encrypt and send: $\text{Enc}_{pk_1}(r'_i) \rightarrow \text{Party 1}$, $\text{Enc}_{pk_2}(s'_i) \rightarrow \text{Party 2}$.
5. Verification: Follow Phases 4-5 from key generation.
 - Public: $R'_i + S'_i = Y_{i+2}$
 - Private: Party 1 checks $r'_i \cdot G = R'_i$, Party 2 checks $s'_i \cdot G = S'_i$

Upon successful verification, parties update their shares: $x_1 \leftarrow x_1 + r'_i$, $x_2 \leftarrow x_2 + s'_i$, and update tracking: $\text{last_seq}[i] \leftarrow \text{seq}_i + 1$, $R_i \leftarrow R'_i$, $S_i \leftarrow S'_i$.

Asynchronous and Independent Operation: Refreshers operate independently without coordination. Multiple refreshers can perform refresh operations concurrently, and their updates commute due to the additive structure. The system's security gradually improves as more refreshers complete refresh operations, and signing continues uninterrupted throughout all refresh operations.

Key Rotation Trigger: Key rotation becomes necessary when the number of honest and participating refreshers falls below an acceptable threshold. This typically occurs when most refreshers have been compromised or have become permanently unavailable, leaving insufficient proactive security margin. The rotation process involves generating a new secret key with a fresh set of refreshers, which may include honest refreshers from the

previous set alongside newly added refreshers. Unlike continuous refresh operations that maintain the same secret key by re-randomizing share splits, key rotation fundamentally changes the underlying secret and requires updating the corresponding public key on-chain or in the relevant trust infrastructure. By extending the time between such rotations through frequent off-chain refresh, our approach significantly reduces the operational costs and complexity associated with on-chain key updates.

3.5. Synchronization Mechanisms

Since refreshers operate independently and asynchronously, each refresher R_i maintains its own monotonically increasing sequence number seq_i for tracking its refresh operations. Sequence numbers across different refreshers are independent; refresher R_1 may be at sequence 100 while refresher R_2 is at sequence 5, reflecting their different participation rates in refresh operations.

Party 1 and Party 2 each maintain a tracking table $\text{last_seq}[i]$ storing the most recent sequence number successfully applied from each refresher R_i . When a refresh message $(i, \text{seq}, R'_i, S'_i, \text{ciphertext})$ is received from refresher i :

- If $\text{seq} \leq \text{last_seq}[i]$: The update has already been applied (or is a replay attack). Discard the message.
- If $\text{seq} = \text{last_seq}[i] + 1$: The update is next in sequence. Process and apply it.
- If $\text{seq} > \text{last_seq}[i] + 1$: The update is out-of-order. Buffer for later application once missing updates arrive, or request retransmission of missing updates from refresher R_i .

After successfully applying an update from refresher i with sequence number seq , parties update: $\text{last_seq}[i] \leftarrow \text{seq}$. This mechanism ensures idempotent application: even if the same refresh message is received multiple times due to network duplication, it is applied exactly once.

The verification mechanisms described in Section 3.3 (Phases 4-6) ensure consistency between Party 1 and Party 2. Both parties apply the same sequence of updates from each refresher, maintaining shares that correctly sum to the secret key x .

4. Security Analysis

This section provides a rigorous security analysis of our key refresh architecture based on the adversary model described in section 3.1.3.

4.1. Confidentiality Under Adversarial Compromise

We now analyze confidentiality of the secret key x under various adversarial compromise scenarios. The fundamental security property is that compromising refreshers alone, or compromising one signing party with any number of refreshers, does not reveal the secret key.

Theorem 1 (Confidentiality): The secret key x remains computationally indistinguishable from a random value in $\mathbb{Z}_q$ for any PPT adversary A that does not simultaneously corrupt both Party 1 and Party 2 within the same refresh epoch.

Proof Sketch: The secret key $x = x_1 + x_2 + \sum x_{i+2}$, where each share is uniformly random. Compromising only refreshers reveals their shares but not x_1 or x_2 , leaving x uniformly distributed. Compromising one signing party (for example Party 1) reveals $x_1^{\text{eff}} = x_1 + \sum r_i$ but not x_2 , keeping $x = x_1^{\text{eff}} + x_2^{\text{eff}}$ uniformly distributed. Only when both signing parties are compromised simultaneously does the adversary learn x .

Corollary 1: Compromising all m refreshers does not compromise the secret key x as long as at least one signing party remains honest.

Proof: This is a direct consequence of Theorem 1. Even with complete knowledge of $\{x_3, x_4, \dots, x_{m+2}\}$, the adversary cannot determine x without learning both x_1 and x_2 .

Practical Implications: These results demonstrate that the security of our system does not depend on the number of honest refreshers. Even if all refreshers are compromised, the secret key remains secure as long as the signing parties remain honest. This is a significant advantage over committee-based approaches where compromising t -of- n committee members compromises the entire secret. In our architecture, refreshers provide operational benefits (proactive security through refresh) but are not critical for fundamental confidentiality of the secret key.

4.2. Proactive Security Guarantees

Proactive security addresses mobile adversaries who compromise different parties across multiple epochs. The key guarantee is that an adversary must compromise both signing parties within a single epoch to reconstruct the secret key. Old split components obtained from previous epochs become useless after honest parties complete refresh and securely erase them. However, the underlying shares (x_1, x_2, x_{i+2}) remain constant and valid across all epochs.

4.2.1. Standard Proactive Security

Theorem 2 (Proactive Security):

Suppose an adversary A compromises Party 1 at time t_1 in epoch e_1 and Party 2 at time $t_2 > t_1$ in epoch $e_2 > e_1$. If at least one honest refresher performs a successful refresh operation between t_1 and t_2 , and honest parties securely erase old share components after refresh, then A cannot reconstruct the secret key x .

Proof Sketch:

At time t_1 , adversary A compromises Party 1 and learns:

$$x_1, \{r_i^{(e_1)}\}_{i=1}^m \quad (\text{epoch } e_1 \text{ split components})$$

Between t_1 and t_2 , at least one honest refresher R_j performs refresh: generates fresh r'_j , computes $s'_j = x_{j+2} - r'_j$, and sends $\Delta r_j = r'_j - r_j$ to Party 1 and $\Delta s_j = s'_j - s_j$ to

Party 2. Honest Party 2 (not yet compromised) applies the update:

$$x_2 \leftarrow x_2 + \Delta s_j$$

$$s_j \leftarrow s'_j$$

Crucially, Party 2 securely erases the old value $s_j^{(e_1)}$. At time t_2 in epoch e_2 , adversary A compromises Party 2 and learns:

$$x_2, \{s_i^{(e_2)}\}_{i=1}^m \quad (\text{epoch } e_2 \text{ split components})$$

The adversary now has:

$$\text{From epoch } e_1 : \quad x_1 \text{ and } \{r_i^{(e_1)}\}$$

$$\text{From epoch } e_2 : \quad x_2 \text{ and } \{s_i^{(e_2)}\}$$

The adversary attempts to reconstruct

$$x = x_1 + x_2 + \sum x_{i+2}.$$

However, for refresher j whose refresh occurred between epochs:

$$r_j^{(e_1)} + s_j^{(e_2)} \neq x_{j+2}$$

because

$$s_j^{(e_2)} = x_{j+2} - r'_j$$

while $r_j^{(e_1)}$ corresponds to old randomness.

The adversary needs either $(r_j^{(e_2)}, s_j^{(e_2)})$ or $(r_j^{(e_1)}, s_j^{(e_1)})$ to reconstruct x_{j+2} . However, $r_j^{(e_2)}$ was only sent to Party 1 (already compromised at t_1 , but Party 1 applies refresh updates after the adversary leaves), and $s_j^{(e_1)}$ was securely erased by Party 2 before time t_2 . Therefore, A cannot obtain matching split components from the same epoch, preventing reconstruction of x_{j+2} and thus x .

4.2.2. Degraded Scenarios

We now analyze scenarios where the refresh mechanism operates under degraded conditions. When the adversary compromises all refreshers except one (say, R_m remains honest), the system continues operating with maintained confidentiality (Theorem 1). If the honest refresher R_m is available during the interval between compromises of the two signing parties, it provides proactive security (Theorem 2). If R_m is unavailable when refresh is needed, the system continues using old splits without achieving proactive security for that epoch, though confidentiality is maintained.

If the adversary compromises all m refreshers, the refresh mechanism effectively fails and no new refresh operations can be trusted. However, confidentiality remains secure as long as one signing party is honest (Corollary 1). The system reverts to static secret sharing, where compromising both signing parties at any time (even in different epochs) would compromise the secret x . Signing

operations continue normally using current shares, maintaining operational availability despite losing proactive security benefits. In this scenario, the key rotation process is required to generate a new secret key with a fresh set of refreshers, which may include surviving honest refreshers from the previous set alongside newly added ones.

Sequential compromise of refreshers across multiple epochs provides no cumulative advantage to the adversary. Since each refresher holds a constant share x_{i+2} throughout its lifetime, compromising refresher R_i in epoch e_1 and again in epoch e_2 yields no additional information beyond the initial compromise. The critical factor for proactive security is whether at least one refresher remains honest between compromises of the two signing parties, not the total number of refresher compromises across epochs.

4.2.3. Comparison with Committee-Based Approaches

Our architecture provides different security-availability tradeoffs compared to committee-based proactive secret sharing [13-17]. In committee-based approaches, compromising t -of- n committee members compromises the entire secret, regardless of refresh. However, as long as fewer than t members are compromised, the system maintains both confidentiality and proactive security.

In our architecture, compromising all refreshers does not compromise confidentiality (by Corollary 1) but eliminates proactive security. This reflects our design goal of separating roles: refreshers provide operational benefits (proactive security) but are not fundamental to confidentiality. The confidentiality guarantee depends solely on the signing parties remaining honest, while proactive security depends on having at least one honest refresher available between compromise events.

This separation enables graceful degradation: the system continues operating securely (for confidentiality) even when all refreshers are compromised, though without proactive security benefits. In contrast, committee-based systems face complete failure once t members are compromised.

4.3. Collusion Analysis

We analyze security when multiple parties collude, sharing their private information to attempt reconstruction of the secret key x . These scenarios are critical for understanding the trust boundaries of our architecture.

4.3.1. Signing Party with Refreshers

Theorem 3 (Collusion Resistance):

For any subset $S \subseteq \{R_1, \dots, R_m\}$ of refreshers, collusion between Party 1 and refreshers in S (or between Party 2 and refreshers in S) does not compromise the secret key x .

This follows directly from Corollary 1 since x_2 is uniformly random and independent of the colluding parties' view, the value $x = x_1 + x_2 + \sum x_{i+2}$ remains uniformly distributed over Z_q from their perspective. The collusion

provides no advantage in recovering x . The symmetric argument applies to collusion between Party 2 and refreshers: they can compute $x_2 + \sum_{i \in S} x_{i+2}$ but cannot determine x without knowing x_1 .

Corollary 2:

Collusion between one signing party and all m refreshers does not compromise the secret key x .

This follows directly from Theorem 3 by setting $S = \{R_1, \dots, R_m\}$. Even if Party 1 colludes with every single refresher, they collectively learn $x_1 + \sum_{i=1}^m x_{i+2} = x - x_2$, but x_2 remains unknown and uniformly random. The same holds for Party 2 colluding with all refreshers.

4.3.2. Both Signing Parties Within Same Epoch

The case where an adversary compromises both Party 1 and Party 2 within the same epoch is straightforward: the adversary obtains x_1^{eff} and x_2^{eff} from the same epoch, which sum to x . The secret key is compromised. This is inherent to any 2-of-2 threshold scheme and represents the fundamental security boundary of our architecture.

Importantly, the presence of refreshers does not change this boundary. Even with m refreshers holding $\{x_3, \dots, x_{m+2}\}$, if both signing parties are compromised simultaneously, the adversary reconstructs $x = x_1^{eff} + x_2^{eff}$ without needing the refreshers' shares. The refreshers provide proactive security by enabling recovery from sequential compromises across different epochs, but cannot protect against simultaneous compromise of both signing parties within a single epoch.

4.4. Denial of Service Resistance

Denial of service (DoS) attacks aim to disrupt system operations without necessarily compromising confidentiality. In our architecture, the ability to operate without requiring all refreshers to participate provides inherent resistance to many DoS attacks. However, a sophisticated adversary might target the remaining honest refreshers to eliminate proactive security capabilities entirely. This section analyzes DoS resistance, with particular focus on attacks against honest refreshers in scenarios where most refreshers have already been compromised or disabled.

4.4.1. DoS Attack Model

We consider an adversary A^{DoS} whose goal is to prevent the system from performing successful refresh operations, thereby eliminating proactive security benefits while potentially allowing continued signing operations. The adversary may employ the following strategies:

Refresher Compromise and Disabling: Compromise refreshers and force them to cease operations or send invalid updates that fail verification.

Network Disruption: Block or delay network communication between honest refreshers and signing parties, preventing delivery of valid refresh updates.

Resource Exhaustion: Overwhelm signing parties with computationally expensive verification operations through flooding attacks.

Targeted Attacks on Honest Refreshers: After compromising $m-1$ refreshers, focus attacks specifically on the last remaining honest refresher to eliminate all proactive security capability.

The critical distinction from confidentiality attacks is that DoS attacks need not compromise secret key material; disrupting refresh operations alone is sufficient to achieve the adversary's goal of preventing proactive security.

4.4.2. Basic DoS Resistance Through Architectural Design

Our architecture provides inherent resistance to several classes of DoS attacks through its design principles:

Graceful Degradation Under Refresher Failure:

As analyzed in Section 4.3.2, the system continues operating even when many refreshers are unavailable or malicious. If refresher R_i is compromised and refuses to send updates or sends invalid updates, it is simply excluded from subsequent operations after verification fails (Phases 4-5 in Section 3.3). Signing continues using that refresher's last valid split components. Compromising fewer than m refreshers does not cause system failure, and the system maintains signing availability regardless of refresher behavior.

No Coordination Requirements: Unlike committee-based approaches [13–17] where DoS against a threshold of committee members prevents refresh, our refreshers operate independently. An adversary cannot prevent refresh by disrupting coordination, as no coordination exists. Each honest refresher can perform refresh operations independently of other refreshers' availability or behavior.

Early Rejection of Invalid Updates: The two-phase verification mechanism (public commitment verification in Phase 4, private decryption verification in Phase 5) allows early rejection of malicious refresh attempts. If a malicious refresher sends commitments R_i, S_i that do not satisfy $R_i + S_i = Y_{i+2}$, all parties immediately detect this in Phase 4 without requiring expensive decryption operations. This limits the computational cost imposed by malicious refreshers to simple elliptic curve point additions.

Key Rotation as Recovery: When the adversary has already compromised most refreshers, DoS attacks against the few remaining honest refreshers become more effective. In such degraded scenarios, key rotation provides recovery: generating a new secret key with a fresh set of refreshers (which may include surviving honest refreshers alongside new ones). Unlike refresh operations that re-randomize splits while maintaining the same key, rotation changes the underlying secret and requires updating the public key on-chain. Our approach extends intervals between such costly rotations by maintaining security through continuous off-chain refresh, requiring rotation only when the honest refresher set becomes critically small.

4.4.3. Critical Scenario: Attacks on Last Honest Refresher

The most concerning DoS scenario, while unlikely in practice due to the availability of key rotation as a recovery mechanism, occurs when an adversary has compromised

$m - 1$ refreshers, leaving only one honest refresher R_j . The adversary can then focus DoS attacks on R_j through network partition (blocking communication between R_j and signing parties), resource exhaustion, or timing attacks concentrated during refresh windows.

If the adversary successfully prevents R_j from communicating during the window between compromising Party 1 (at time t_1) and Party 2 (at time t_2), no valid refresh occurs between compromises. By Theorem 2, proactive security fails and the adversary may reconstruct x from shares obtained in different epochs. However, confidentiality is maintained by Theorem 1 as long as one signing party remains honest at any given time.

DoS against the last honest refresher eliminates proactive security but does not directly compromise confidentiality. The system reverts to static secret sharing: the secret remains secure unless both signing parties are compromised simultaneously, though compromising both at any time (even in different epochs) would compromise x .

5. Comparison and Performance Analysis

This section compares our approach with existing key refresh schemes and analyzes communication and computational complexity. We focus on the key architectural differences and performance characteristics that distinguish our approach from prior work.

5.1. Comparison with Existing Approaches

Table 1 provides a comparative overview of our approach against three categories of existing work: single-dealer refresh (He et al. [18]), offline refresh (Kondi et al. [19]), and committee-based dynamic proactive secret sharing (CHURP [13], DyCAPS [14], and related works [15-17]).

Key Observations:

Coordination-Free Operation: Our approach and He et al.’s single-dealer approach avoid coordination overhead, enabling continuous refresh without disrupting operations. Committee-based and offline refresh approaches require coordination among multiple parties, which can delay or complicate refresh operations.

Fault Tolerance: Our architecture tolerates unavailability or compromise of any subset of refreshers while maintaining operation. He et al.’s approach has a single point of failure (the dealer), while committee-based approaches require at least t honest members. Our adversarial threshold of $m - 1$ (to degrade to minimal state) is higher than the $n - t$ threshold in committee systems.

Partial Refresh Capability: Unlike other approaches that require all designated parties to participate, our system allows any subset of refreshers to perform refresh independently. This provides operational flexibility and robustness against temporary failures.

Role Separation: Our architecture uniquely separates signing and refreshing roles. Signing parties never participate in refresh, and refreshers never participate in signing. This simplifies each role and enables independent optimization.

5.2. Communication Complexity

We analyze the communication costs of each protocol phase, measuring in terms of point-to-point messages and broadcast messages among $m + 2$ total parties (2 signing parties + m refreshers).

Key Generation (Section 3.3):

- **Phase 1:** Each party broadcasts Y_j . Total: $O(m)$ broadcasts
- **Phase 2:** Party 1 and Party 2 broadcast encryption keys. Total: $O(1)$ broadcasts
- **Phase 3:** Each refresher sends encrypted splits to both parties. Total: $O(m)$ point-to-point messages
- **Phase 4-6:** Verification phases involve $O(m)$ computations but minimal additional communication

Total key generation: $O(m)$ broadcasts + $O(m)$ point-to-point messages. This is linear in the number of refreshers and does not require quadratic all-to-all communication.

Key Refresh (Section 3.4): Each refresher R_i that participates in refresh performs:

- Broadcast commitments $(R_i, S_i, \text{sequence number})$: $O(1)$
- Send encrypted updates to Party 1 and Party 2: $O(1)$ point-to-point messages

Total per-refresher cost: $O(1)$. If k refreshers participate in a given refresh round, total cost is $O(k)$. Crucially, refreshers operate independently—they do not communicate with each other, eliminating $O(k^2)$ coordination overhead.

Signing (Section 3.3, end): Signing involves only Party 1 and Party 2 using standard 2-of-2 threshold signature protocols. Communication complexity depends on the chosen protocol (e.g., CGGMP [5] provides one-round signing after preprocessing). Refreshers do not participate. Total: $O(1)$ between two parties.

Comparison with Committee-Based Approaches: Committee-based DPSS protocols [13–17] typically require $O(n^2)$ communication during refresh, as committee members must perform distributed key generation or resharing protocols that involve all-to-all communication. Even optimized protocols like CHURP [13] require at least $O(n \cdot t)$ communication where t is the threshold. Our $O(k)$ refresh communication (where k is the number of participating refreshers, and k can be as small as 1) represents a significant reduction, particularly when only a subset of refreshers performs refresh in a given round.

Property	Our Approach	He et al. [18]	Kondi et al. [19]	Committee-based [13–17]
Trust Model	2-of-2 signing parties + m independent refreshers	2-of-2 signing parties + single trusted dealer	t -of- n threshold among all parties	t -of- n threshold among committee members
Coordination Required	None	Dealer must broadcast	All parties coordinate	Committee members coordinate
Single Point of Failure	No	Yes (dealer)	No	No
Partial Refresh	Yes (any subset)	No (dealer must be available)	No (all parties must participate)	No (requires t members)
Disrupts Signing	No	No	Yes	Potentially
Adversarial Threshold (Degradation)	$m - 1$ refreshers	1 (dealer)	t parties	$n - t$ members
Communication (Refresh)	$O(1)$ per refresher	$O(1)$ broadcast	$O(n^2)$	$O(n^2)$

Table 1: Comparison of Key Refresh Approaches

5.3. Computational Complexity

We analyze computational costs in terms of expensive cryptographic operations: elliptic curve point operations (additions and scalar multiplications) and encryption/decryption operations.

Key Generation:

- Each party: 1 scalar multiplication to compute $Y_j = x_j \cdot G$
- All parties: m scalar multiplications to verify $\{R_i, S_i\}$ for each refresher
- Party 1 and Party 2: m decryptions and m scalar multiplications for verification

Total per signing party: $O(m)$ scalar multiplications + $O(m)$ decryptions.

This scales linearly with m . Since verification operations are independent for each refresher, they can be parallelized across multiple CPU cores, significantly reducing wall-clock time. Modern multi-core processors can verify large numbers of refreshers efficiently, making the practical limit on m determined by operational considerations (ensuring refresher independence and managing deployment) rather than computational constraints.

Key Refresh: When k refreshers participate in refresh:

- Each refresher: 2 scalar multiplications (compute R'_i, S'_i) + 2 encryptions
- Signing parties: k decryptions + k scalar multiplications (verify commitments)

Total per signing party: $O(k)$ operations.

Since k can be as small as 1, refresh operations can be very lightweight. Even with all m refreshers participating, the cost remains $O(m)$, which is manageable for practical m values.

Signing: Computational cost depends on the 2-of-2 threshold signature protocol used (independent of m). Modern protocols like CGGMP [5] require a small constant number of scalar multiplications per signature. Our architecture adds no overhead to the signing operation.

Scalability Analysis: The computational complexity scales linearly with m in the worst case (all refreshers participating). However, several factors make the system practical:

- Verification operations (scalar multiplications, decryptions) are easily parallelizable across multiple CPU cores

- Modern elliptic curve implementations (e.g., secp256k1) perform scalar multiplication in under 1ms on commodity hardware
- Partial refresh allows spreading the computational load over time—not all refreshers need to refresh simultaneously
- Early rejection in Phase 4 (public commitment verification) prevents expensive decryption for invalid updates

5.4. Practical Considerations

Deployment Scenarios:

Our architecture is particularly well-suited for scenarios where:

High Availability Required: Signing operations must continue with minimal latency regardless of refresh activity. The role separation ensures refresh never blocks signing.

Distributed Trust: Trust can be distributed across multiple independent third-party refreshers rather than concentrated in a single entity or coordinated committee.

Operational Flexibility: Some refreshers may be temporarily unavailable due to maintenance, network issues, or geographic restrictions. The system continues operating with available refreshers.

MPC Wallet Applications: User-controlled Party 1 combined with network-operated Party 2 (threshold signature network) and independent refreshers provides user sovereignty while maintaining security through proactive refresh.

6. Discussion and Future Work

This section discusses limitations of the current approach and promising directions for future research, with particular focus on hybrid models where refreshers can optionally participate in signing operations.

6.1. Limitation

Secure Erasure Assumption: Like all proactive secret sharing schemes [9, 10, 13–19], our current design’s security depends on honest parties securely erasing old cryptographic material after refresh. While standard in the literature, secure deletion remains challenging due to caching, swap files, and persistent storage. Hardware security modules can strengthen this assumption but add complexity. However, the hybrid model discussed in the next

section, where refreshers participate directly in signing, significantly weakens this requirement: if at least one refresher participates in signing and remains uncompromised, proactive security is achieved even if signing parties fail to achieve perfect secure deletion.

6.2. Hybrid Model: Refreshers Participating in Signing

An important direction for future research involves allowing refreshers to actively participate in signing when available, while maintaining graceful degradation when unavailable. This hybrid approach could increase the effective signing threshold beyond 2-of-2, providing stronger security.

Importantly, this hybrid model significantly weakens the secure erasure assumption. In the current design, cross-epoch security depends on signing parties securely erasing old split components. In the hybrid model, if at least one refresher participates directly in signing and maintains its share x_{i+2} securely, the adversary must compromise that refresher in addition to both signing parties, even across different epochs. This provides proactive security even if signing parties fail to achieve perfect secure deletion, as long as the participating refresher remains uncompromised.

Basic Concept: In the hybrid model, when refresher R_i is available, it participates directly in a multi-party threshold signature using its share x_{i+2} . The signing protocol becomes a $(2 + k)$ -party signature where k is the number of available refreshers. When R_i is unavailable, its split components (r_i, s_i) are provided to Party 1 and Party 2, who incorporate them in their secret keys as in our current design. Because Party 1 and Party 2 now know its share x_{i+2} (by combining $r_i + s_i$), R_i is eliminated from further signing. This maintains fault tolerance while potentially increasing security when refreshers participate.

Technical Challenges:

- **Secure Distribution of Split Components:** When a refresher is unavailable, how should its split components be securely provided to signing parties? Options include pre-distribution (reduces security benefit), escrow services (new trust assumptions), distributed storage (availability vs. confidentiality tradeoffs), or threshold decryption (coordination overhead). Each presents different security-availability-complexity tradeoffs.
- **Ensuring Continuous Availability:** The system must guarantee that split components remain accessible even when refreshers are offline for extended periods. Challenges include: where to store components persistently, how to keep them synchronized with refresh operations, and how to control access without compromising security.
- **Security Analysis:** The hybrid model requires careful security analysis. An adversary might strategically prevent refresher participation via denial-of-service to

force split-component usage. The effective security threshold becomes variable and depends on refresher participation rates. Additionally, refresher participation in signing might reveal information affecting proactive security properties.

Potential Solutions: Several promising approaches exist:

1. Verifiable timed-release encryption using verifiable delay functions could enable time-based decryption of split components with optional early release by refreshers.
2. Secret sharing schemes encoding reconstruction policies directly into shares could specify conditions for different subsets to reconstruct secrets.
3. Secure enclaves (Intel SGX, AMD SEV, ARM TrustZone) could store encrypted split components and enforce access policies through attestation, releasing splits to signing parties only when refreshers prove unavailability. This provides strong confidentiality (splits protected even if the host is compromised) and guaranteed availability without requiring additional trusted parties.
4. Blockchain-based availability using distributed ledgers could provide strong availability guarantees for encrypted split components.
5. Optimistic protocols could assume refresher availability and fall back to split-component-based signing only when necessary, maintaining simplicity for the common case.

6.3. Conclusion

We have presented a novel architecture for proactive key refresh that separates signing and refreshing roles through multiple independent third-party refreshers. Key advantages include: elimination of coordination overhead, graceful degradation under partial failures, higher adversarial thresholds ($m-1$ refreshers must be compromised to reach minimal state vs. $n-t$ in committee systems), and continuous signing availability during refresh. Our security analysis demonstrates that the secret key remains secure as long as one signing party is honest, regardless of refresher compromises, and that proactive security operates even with minimal honest refresher participation. The $O(k)$ communication complexity for k participating refreshers compares favorably to $O(n^2)$ for committee-based approaches. The hybrid model proposed in Section 6.2 offers a promising direction for enhancing security through optional refresher participation in signing while maintaining operational flexibility. As threshold cryptography becomes increasingly critical for securing digital assets and distributed systems, our architecture provides a practical foundation for proactive security that adapts to evolving threats without disrupting operations.

References

- [1] Y. Desmedt and Y. Frankel, "Threshold Cryptosystems," in CRYPTO 1989.
- [2] Y. Lindell, "Fast Secure Two-Party ECDSA Signing," in CRYPTO 2017.
- [3] R. Gennaro and S. Goldfeder, "Fast Multiparty Threshold ECDSA with Fast Trustless Setup," in ACM CCS 2018, ePrint 2019/114.
- [4] R. Gennaro and S. Goldfeder, "One Round Threshold ECDSA with Identifiable Abort," ePrint 2020/540.
- [5] R. Canetti, N. Makriyannis, and U. Peled, "UC Non-Interactive, Proactive, Threshold ECDSA with Identifiable Aborts," in ACM CCS 2020, ePrint 2020/492.
- [6] O. Goldreich, S. Micali, and A. Wigderson, "How to Play Any Mental Game," in STOC 1987.
- [7] P. Feldman, "A Practical Scheme for Non-interactive Verifiable Secret Sharing," in FOCS 1987.
- [8] D. Cooper, S. Santesson, S. Farrell, S. Boeyen, R. Housley, and W. Polk, "Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile," RFC 5280, 2008.
- [9] A. Herzberg, S. Jarecki, H. Krawczyk, and M. Yung, "Proactive Secret Sharing or: How to Cope with Perpetual Leakage," in CRYPTO 1995.
- [10] D. Schultz, B. Liskov, and M. Liskov, "MPSS: Mobile Proactive Secret Sharing," in ACM PODC 2008.
- [11] J. Baron, K. Eldefrawy, J. Lampkins, and R. Ostrovsky, "Communication-Optimal Proactive Secret Sharing for Dynamic Groups," in ACNS 2015.
- [12] K. Eldefrawy, T. Lepoint, and A. Leroux, "Communication-Efficient Proactive Secret Sharing for Dynamic Groups with Dishonest Majorities," in ACNS 2020, ePrint 2019/1383.
- [13] S.K.D. Maram, F. Zhang, L. Wang, A. Low, Y. Zhang, A. Juels, and D. Song, "CHURP: Dynamic-Committee Proactive Secret Sharing," in ACM CCS 2019.
- [14] B. Hu, Z. Zhang, H. Chen, Y. Zhou, H. Jiang, and J. Liu, "DyCAPS: Asynchronous Dynamic-committee Proactive Secret Sharing," ePrint 2022/1169, Chinese Journal of Electronics 2025.
- [15] B. Hu, J. Liu, Z. Lu, Q. Tang, Z. Xiang, and Z. Zhang, "Optimistic Asynchronous Dynamic-committee Proactive Secret Sharing," ePrint 2025/880.
- [16] A. Bandarupalli, X. Ji, S. Jog, A. Kate, C. Liu-Zhang, and Y. Song, "Computationally and Communication Efficient Batched Asynchronous DPSS from Lightweight Cryptography," ePrint 2025/1631.
- [17] T. Yurek, Z. Xiang, Y. Xia, and A. Miller, "Long Live The Honey Badger: Robust Asynchronous DPSS and its Applications," in USENIX Security 2023, ePrint 2022/971.
- [18] Y. He, Y. Kadera, Y. Nogami, and S. Huda, "Role-Based Efficient Proactive Secret Sharing with User Revocation," Cryptography, vol. 9, no. 4, p. 80, 2025.
- [19] Y. Kondi, B. Magri, C. Orlandi, and O. Shlomovits, "Refresh When You Wake Up: Proactive Threshold Wallets with Offline Devices," in IEEE S&P 2021, ePrint 2019/1328.
- [20] B. Koziel, S.D. Gordon, and C. Gentry, "Fast Two-party Threshold ECDSA with Proactive Security," in ACM CCS 2024, ePrint 2024/1831.
- [21] J. Camenisch and A. Lysyanskaya, "Dynamic Accumulators and Application to Efficient Revocation of Anonymous Credentials," in CRYPTO 2002.
- [22] A. Bessani, M. Correia, B. Quaresma, F. André, and P. Sousa, "DepSky: Dependable and Secure Storage in a Cloud-of-Clouds," in EuroSys 2011.
- [23] A. Shamir, "How to Share a Secret," Communications of the ACM, vol. 22, no. 11, pp. 612-613, 1979.
- [24] C. Komlo and I. Goldberg, "FROST: Flexible Round-Optimized Schnorr Threshold Signatures," in Selected Areas in Cryptography 2020.
- [25] C.G. Günther, S. Das, and L. Kokoris-Kogias, "Practical Asynchronous Proactive Secret Sharing and Key Refresh," ePrint 2022/1586.
- [26] ZenGo Research Team. <https://zengo.com>
- [27] Fireblocks. "What is MPC (Multi-Party Computation)?" <https://www.fireblocks.com/what-is-mpc>
- [28] P. Das, A. Erwig, S. Faust, P. Lehwalder, J. Loss, Z. Qu, and S. Riahi, "BIP32-Compatible Threshold Wallets," in ACM AsiaCCS 2025, ePrint 2023/312.
- [29] P. Das, A. Erwig, M. Meyer, and P. Struck, "Efficient Post-Quantum Secure Deterministic Threshold Wallets from Isogenies," in ACM AsiaCCS 2024.
- [30] B. Gao, R. Zhang, Y. Xiao, and H. Zou, "NEST: Strong Key-Insulated Password-Based Shared-Custodial Blockchain Wallets," in Inscrypt 2024, Springer LNCS, 2025.
- [31] Gennaro, R., Jarecki, S., Krawczyk, H., & Rabin, T. (2007). Secure distributed key generation for discrete-log based cryptosystems. Journal of Cryptology, 20(1), 51-83.
- [32] Hazay, C., & Lindell, Y. (2010). Efficient Secure Two-Party Protocols: Techniques and Constructions. Springer.